TEMAS SELECTOS DE PROGRAMACIÓN CONCURRENTE

Erlang/OTP en el mundo real

Cuándo usar Cowboy, Poolboy, epgsql, recon y Prometheus

Plataforma de referencia: Raspberry Pi Compute Module 5 · Debian 13 · OTP 27

René Solis R. · Tecnológico Nacional de México, campus Tijuana

Basado en: gist.github.com/IoTeacher/3c3607e48bce473de9bfa6d400dec6ff

CONTENIDO

Agenda

1. **Fundamentos**
   BEAM en el CM5 y árbol de supervisión
2. **Capa web**
   Cowboy, Ranch, WebSocket frente a SSE
3. **Datos**
   Poolboy, epgsql y la decisión ETS / PostgreSQL / Mnesia
4. **Operación**
   recon, Prometheus, releases y systemd
5. **Producción**
   Afinado de VM y SO, TLS y secretos
6. **Decisión**
   Guía rápida: qué herramienta para qué problema

Por qué BEAM en un Compute Module 5

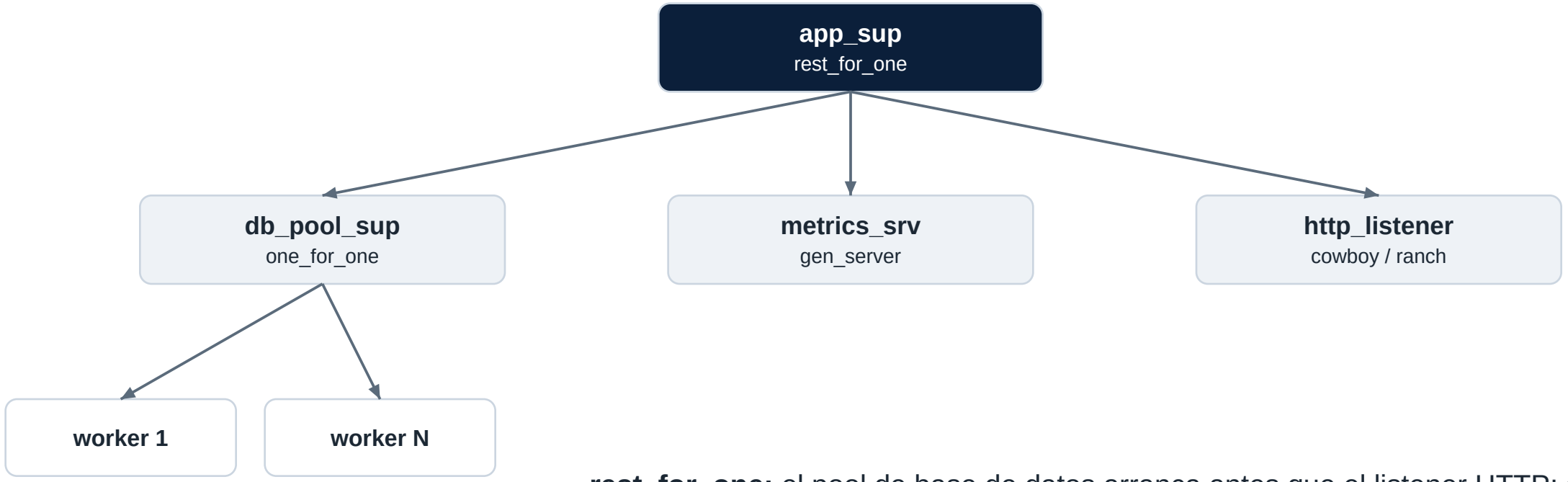
- **Procesos, no hilos del SO:** millones de procesos ligeros con heap propio; aislamiento de fallos por diseño.
- **Un scheduler por núcleo físico:** el CM5 tiene 4 Cortex-A76, así que `+S 4:4`.
- **JIT nativo** (desde OTP 24, también en aarch64): menos costo de interpretación en el mismo hardware.
- **8 GB de RAM:** unos 5.5 GB para la app; el resto para SO y caché de disco.
- **Let it crash:** se asume que los procesos fallan y se les reinicia desde un supervisor.

Hardware de referencia

CPU	**4 × Cortex-A76**
RAM	**8 GB**
Arquitectura	**aarch64**
SO	**Debian 13 (Trixie)**
Erlang	**OTP 27.3.x + JIT**
Build	**rebar3 + relx**

> No se asume hardware en el código: el desarrollo puede ocurrir en macOS o Linux.

Árbol de supervisión



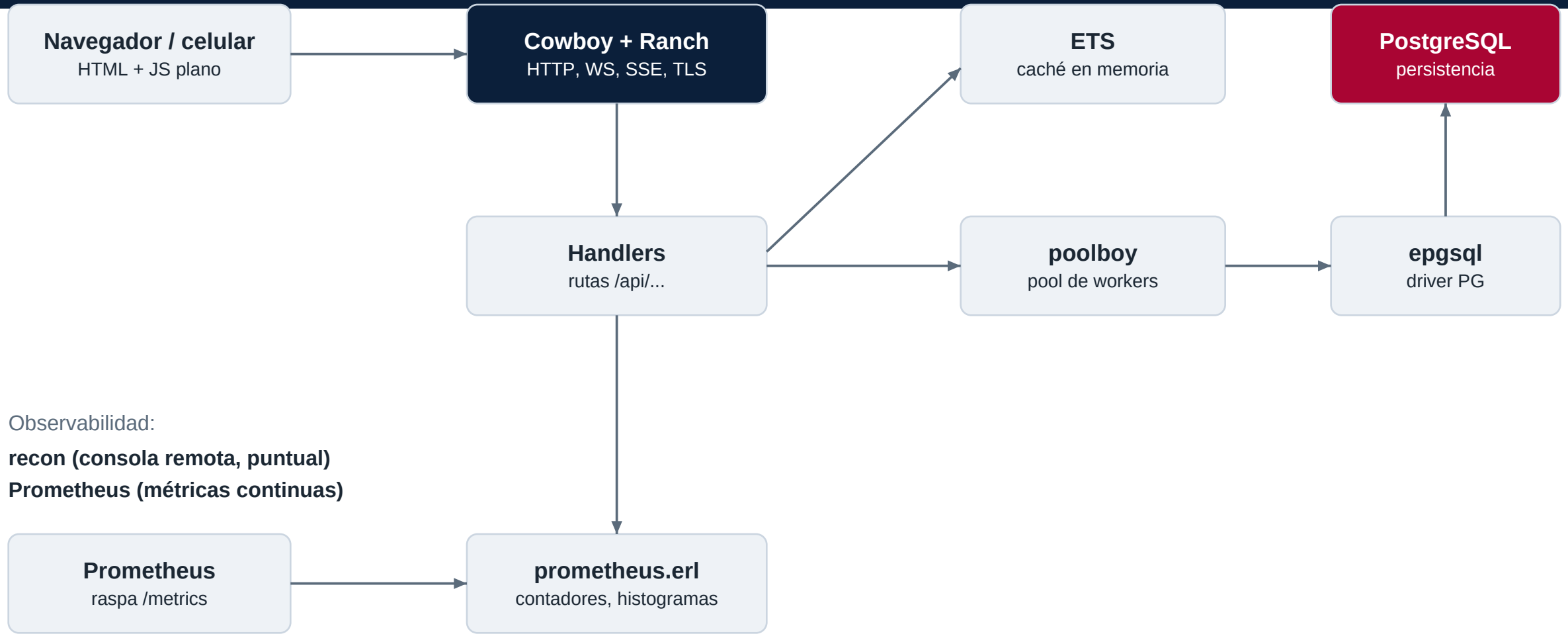
epgsql + poolboy

- **rest_for_one:** el pool de base de datos arranca antes que el listener HTTP; si cae el pool, se reinicia también lo que depende de él.
- **one_for_one:** workers independientes; la caída de uno no afecta a los demás.

> Todo proceso expone `start_link/0` y cuelga de un supervisor. Evitar `spawn` suelto.

FUNDAMENTOS

Arquitectura del stack completo



Observabilidad:

recon (consola remota, puntual)

Prometheus (métricas continuas)

Cowboy 2.12 y Ranch 2.1

Cuándo usarlo

- API JSON y archivos estáticos (`cowboy_static`).
- Tiempo real con muchas conexiones: WebSocket y SSE.
- TLS directo en el nodo.
- HTTP/1.1 y HTTP/2.

Cuándo no

- Necesitas un framework completo (ORM, sesiones, plantillas): Nova o Phoenix.
- Solo necesitas un *cliente* HTTP: gun o hackney.

Modelo de procesos: Ranch acepta conexiones y Cowboy lanza un proceso por conexión y por request. Si un handler falla, muere solo ese proceso; el resto del servidor sigue atendiendo.

Cowboy en la práctica

```
Dispatch = cowboy_router:compile([{'_', [
    {"/api/temp", temp_handler, []},
    {"/ws",       temp_ws_handler, []},
    {"/[...]",    cowboy_static, {priv_dir, mi_app, "www"}}
]}]),
{ok, _} = cowboy:start_clear(http, [{port, 8080}],
                             #{env => #{dispatch => Dispatch}}).
```

- El **router** se compila una sola vez; las rutas apuntan a módulos con callback `init/2`.
- `start_clear` es HTTP plano; para HTTPS se usa `start_tls` con certificado.
- La UI exportada desde Stitch se coloca en `priv/www` y se sirve con `cowboy_static`.
- El backend solo sirve datos y archivos; sin frameworks JS pesados.

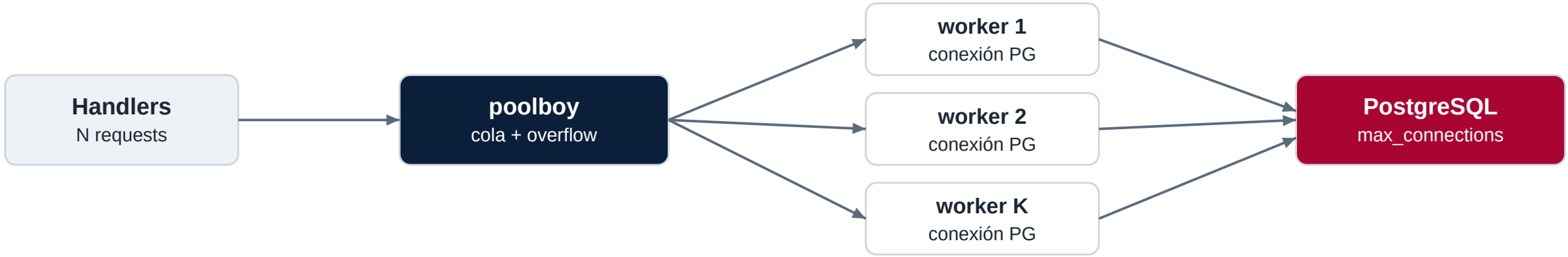
WebSocket frente a SSE

Criterio	WebSocket	SSE
Dirección	Bidireccional	Servidor hacia cliente
Protocolo	Upgrade a `ws://`	HTTP normal, `text/event-stream`
Reconexión	La implementa la app	Automática en el navegador
Proxies / firewalls	A veces problemático	Atraviesa sin problema
Ideal para	Chat, control interactivo, juegos	Dashboards de sensores, notificaciones

> **Regla práctica:** si el cliente solo observa (un panel de temperatura en el celular), SSE es más simple. Si también envía comandos en el mismo canal, WebSocket.

DATOS

Poolboy 1.5 y epgsql 4.7



- Se hace pool para **proteger el recurso**, no por el costo de crear procesos (que es mínimo en BEAM).
- Una conexión epgsql ejecuta una consulta a la vez; el pool da concurrencia acotada.
- Siempre consultas parametrizadas (`$1, $2`): nunca concatenar strings, para evitar inyección SQL.

Dónde guardar los datos

Opción	Persistencia	Ideal para	Limitación
ETS	Solo memoria, se pierde al caer el nodo	Caché, contadores, estado de lectura intensiva	No distribuido; límite = RAM
PostgreSQL (epgsql)	Disco, transaccional	Datos de negocio, consultas relacionales	Latencia de red; requiere pool
Mnesia	Disco o RAM, distribuida	Metadatos de clúster, datos pequeños	No escala bien con grandes volúmenes ni en particiones de red

Decisión del proyecto: PostgreSQL como base principal; Mnesia no se usa como base principal. ETS actúa como caché delante de PostgreSQL.

OPERACIÓN

Observabilidad: recon y Prometheus

recon 2.5 · diagnóstico puntual

- Responde: *¿qué pasa ahora?*
- Procesos con cola creciente:
  `recon:proc_count(message_queue_len, 5)`
- Fugas de binarios:
  `recon:bin_leak(5)`
- Trazas seguras: `recon_trace:calls/3`

prometheus.erl 4.11 · tendencia

- Responde: *¿cómo ha evolucionado?*
- Contadores, gauges, histogramas y summaries.
- Endpoint `/metrics` que Prometheus raspa.
- Alertas y gráficas (latencia p95, req/s) en Grafana.

En el CM5 no hay GPU: **Observer solo en modo remoto**. Recon corre en la consola sin carga adicional. Se complementan, no se sustituyen.

Releases con relx y servicio systemd

```
$ rebar3 release                  # _build/prod/rel/

[Service]
WorkingDirectory=/opt/hello_erlang
LimitNOFILE=524288
LimitNPROC=32768
Environment=HOME=/opt/hello_erlang
```

- El release incluye su propio runtime (ERTS), `vm.args` y `sys.config`.
- **systemd** reinicia el nodo completo si el proceso muere; el supervisor OTP cubre los procesos internos.
- Los límites de systemd deben coincidir con `limits.d`.

> **Corrección al gist:** `LimitNPROC` estaba en 32088 y `nproc` en limits.d en 32768; se unificó a 32768.

Afinado de la VM y del SO

Flag	Efecto	Por qué importa
`+S 4:4`	4 schedulers en 4 núcleos	Nunca superar los núcleos físicos; solo añade contención
`+P 1048576`	Tope de procesos	Muchos procesos baratos; el límite real suele ser el del SO
`+Q 65536`	Tope de puertos y sockets	Cada conexión TCP consume un puerto
`+hmqd off_heap`	Mensajes fuera del heap	Menos pausa de GC con colas de mensajes grandes
`+zdbbl 65536`	Buffer de distribución (KB)	Evita bloqueos en comunicación entre nodos
`nofile 524288`	Descriptores de archivo	Un socket = un descriptor; sin esto, falla antes la conexión que la VM

TLS y manejo de secretos

```
{ok, _} = cowboy:start_tls(https,
    [{port, 8443},
     {certfile, "priv/ssl/cert.pem"},
     {keyfile,  "priv/ssl/key.pem"}],
    #{env => #{dispatch => Dispatch}}).
```

- Certificado **autofirmado** para desarrollo y pruebas; uno emitido por una CA para producción.
- **Nada de cookies, contraseñas ni certificados reales en el repositorio.**
- Usar `sys.config` o variables de entorno con valores de ejemplo.
- Archivos `.pem` y `.key` en `.gitignore`.

> Todo lo que se sube a un repositorio público puede ser indexado y copiado, aunque se borre después.

DECISIÓN

Guía rápida de decisión

Necesito...	Uso
API REST o servir archivos estáticos	Cowboy
Empujar datos al navegador en tiempo real	Cowboy con SSE (solo lectura) o WebSocket (bidireccional)
Persistencia relacional	epgsql + poolboy sobre PostgreSQL
Caché en memoria muy rápido	ETS
Saber por qué el nodo está lento ahora	recon
Gráficas y alertas históricas	prometheus.erl + Grafana
Estado por entidad (sensor, sesión)	Un gen_server o gen_statem por entidad, bajo supervisor
Cliente HTTP saliente	gun o hackney (fuera del stack; requiere aprobación)

Conclusiones

- Elegir cada pieza por el problema que resuelve, no por costumbre.
- Pool para proteger recursos; supervisores para contener fallos.
- Medir siempre: Prometheus para la tendencia, recon para el diagnóstico.
- Reproducibilidad: releases, límites del SO coherentes y cero secretos en el repo.

REFERENCIAS

ninenines.eu/docs (Cowboy, Ranch, gun)

github.com/ferd/recon · github.com/deadtrickster/prometheus.erl

github.com/devinus/poolboy · github.com/epgsql/epgsql

rebar3.org · erlang.org/doc

Gist base: gist.github.com/IoTeacher/3c3607e48bce473de9bfa6d400dec6ff